Administering workPulse

A guide for system administrators

Shah Investors Home Limited

Every picture here is the real application, and the red numbers on it match the numbered notes beside it — where the text says 1, look for the 1 in the picture.

Two things to take from this guide.
First, workPulse owns less than you would expect: identity belongs to Synapse, the org belongs to Frappe HR, and most “fix it here” instincts are wrong. Second, being an administrator is not the same as having every permission — there are screens you cannot open, on purpose, and that is the system working.

WHAT IS IN HERE

WHAT YOU OWN

1. Three masters, one join

ACCESS

2. Who can reach what
3. Mapping a Synapse role
4. Two things that will catch you out
5. Giving somebody a shorter menu

GOVERNANCE

6. What no control covers
7. Creating the controls

OVERSIGHT

8. MIS
9. Evidence for HR
10. Command Centre
11. Screens you cannot see — Flags
12. ...and the shadow quarter

RUNNING IT

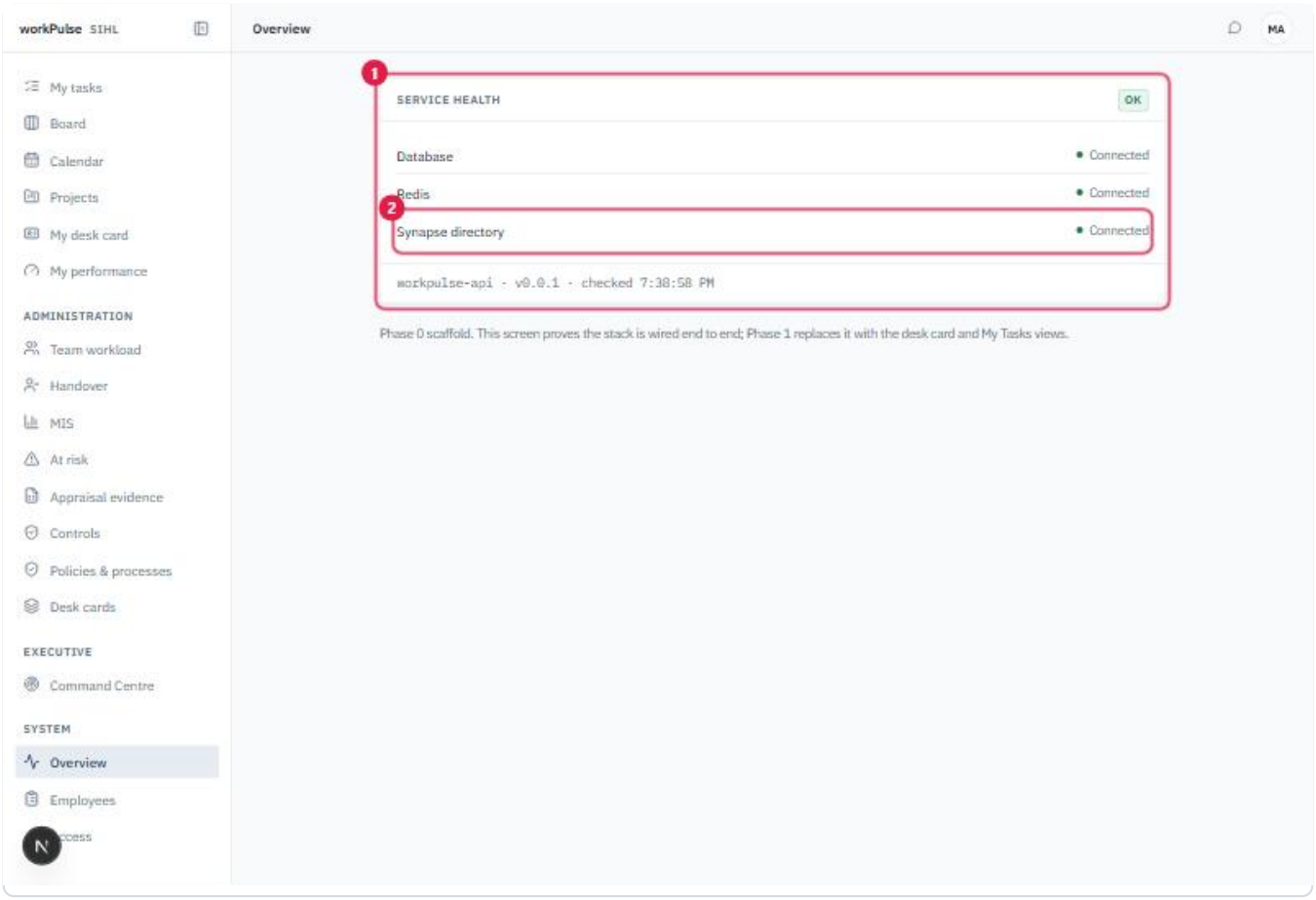
13. Jobs, and what they do
14. Known gaps

Screens captured as a system administrator on an internal system, including the two that refused the role.

WHAT YOU OWN

1. Three masters, one join

Start here, because it decides what you can and cannot fix inside workPulse. Synapse owns identity and roles. Frappe HR owns the org — people, departments, reporting lines. workPulse owns the join between them, and nothing else about either.



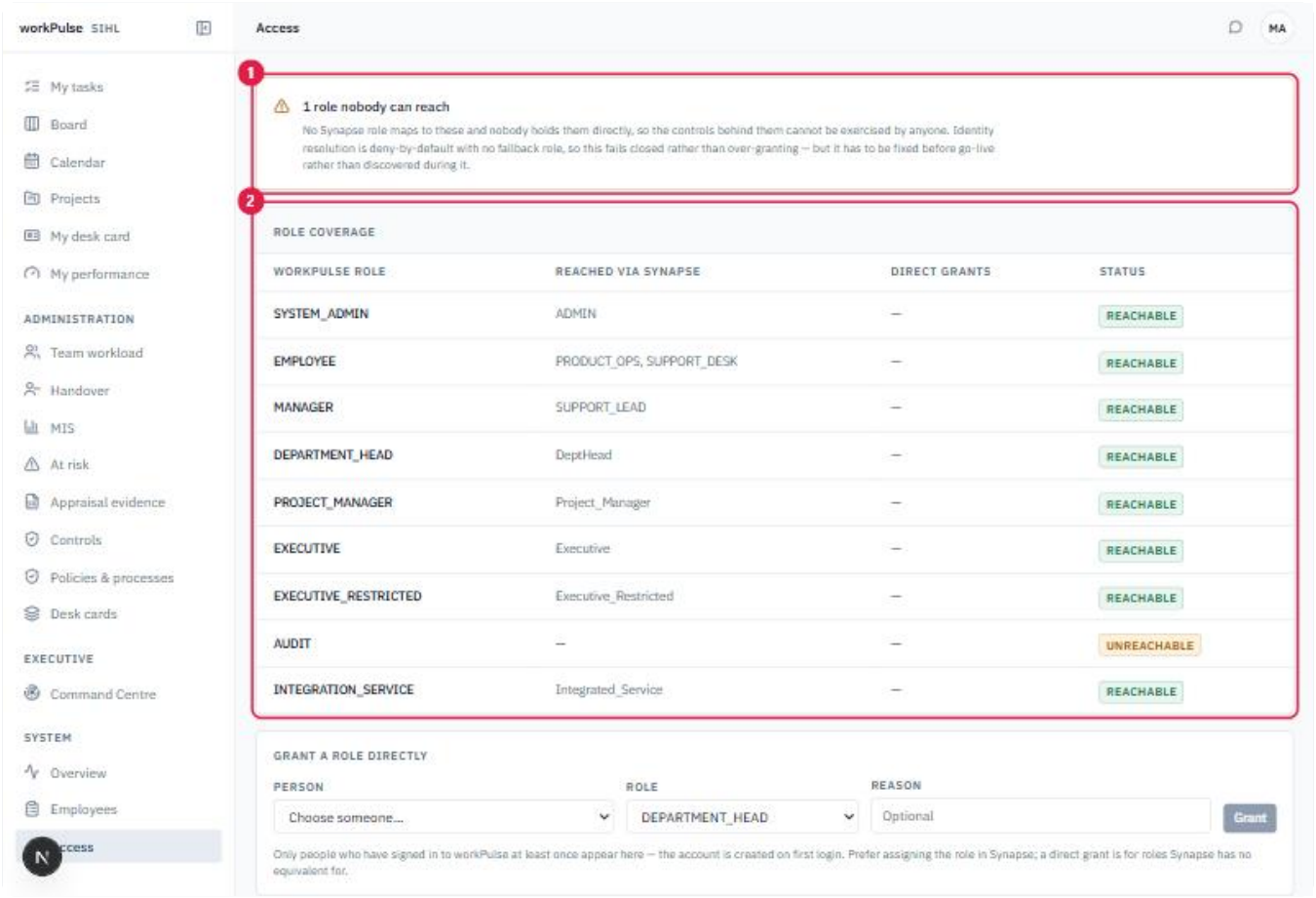
1. Service health. Database and Redis are workPulse's own; if either is down, nothing works.
2. The Synapse directory is a hard dependency, not a nicety. Identity is re-checked against it on every single request, so if this says anything but connected, workPulse is authenticating nobody.

> *The practical consequence: when somebody's department is wrong, fix it in Frappe HR. When their role is wrong, fix it in Synapse. Editing the copy here would be overwritten by the next sync, which is why there is no screen that lets you.*

ACCESS

2. Who can reach what

Access is where you spend most of your time. Every workPulse role, what reaches it, and — the column that matters — whether anybody can reach it at all.



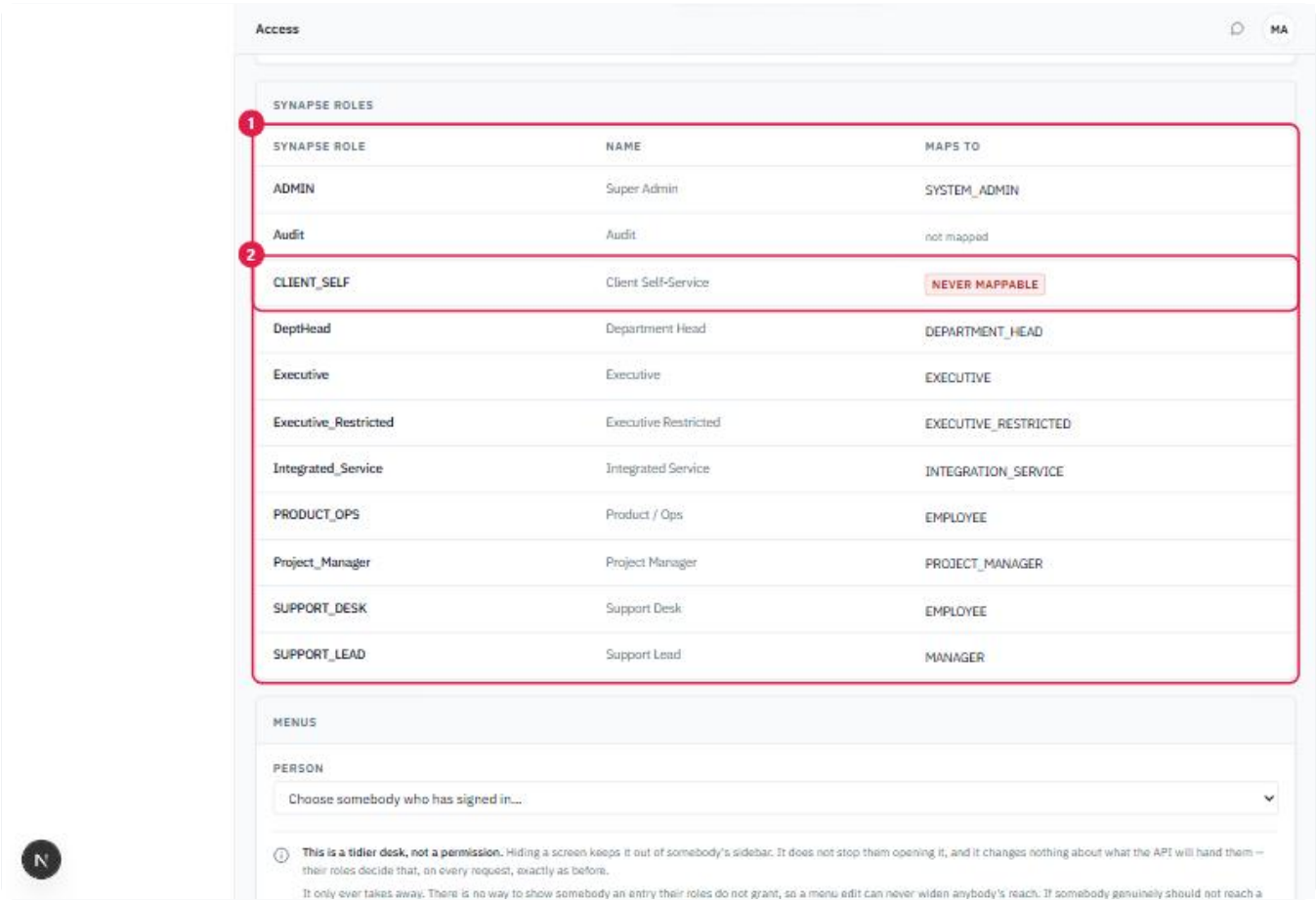
1. A role nobody can reach. No Synapse role maps to it and nobody holds it directly, so the controls behind it cannot be exercised by anyone. This fails closed rather than over-granting, but it has to be fixed before go-live rather than discovered during it.
2. The coverage table. Read it top to bottom before any release.

> *Identity resolution is deny-by-default with no fallback role. A user whose Synapse roles map to nothing is refused entry entirely — they do not quietly become an employee.*

ACCESS

3. Mapping a Synapse role

A mapping says what a Synapse role means in workPulse. It is done once per role, org-wide — not per person. Assigning the role to somebody happens in Synapse, and workPulse re-reads it on every request, so revoking takes effect on their next click rather than when their session expires.



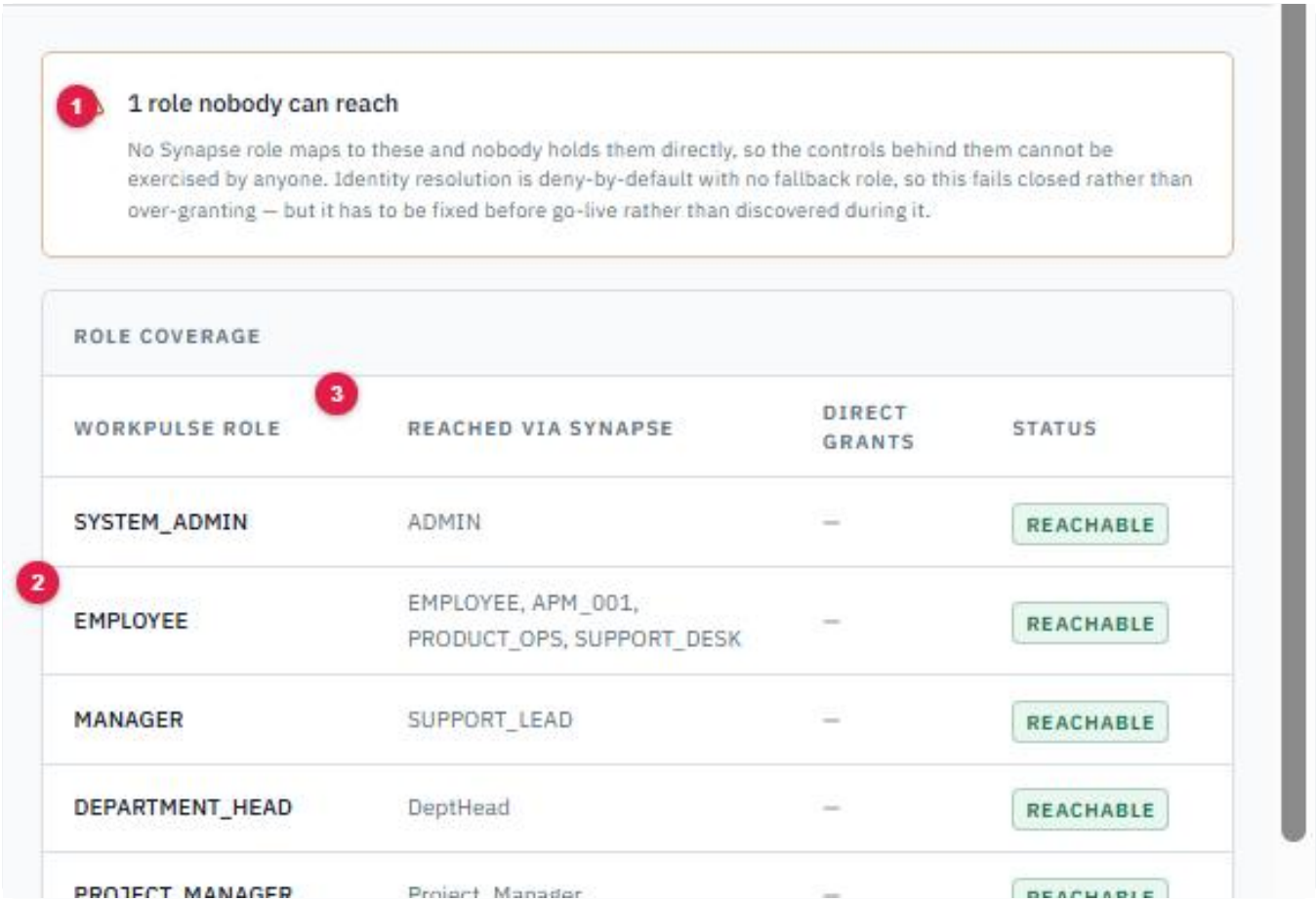
1. Every Synapse role that actually exists, and what it maps to. Mappings are explicit rows, never name matching: inferring that “SIHL_MANAGER” means MANAGER would break silently the first time somebody renamed a role.
2. CLIENT_SELF can never be mapped, at any privilege level. Synapse authenticates external clients as well as staff — mapping it even to the lowest role would hand customers workPulse accounts and desk cards.
3. A role with no mapping grants nothing. That is a decision to make deliberately, not a gap to leave.

> *A direct grant is the exception, for roles Synapse has no equivalent for — notably Audit, which has to sit outside the line hierarchy. Prefer creating the role in Synapse and mapping it: that survives people joining and leaving, which a per-person grant does not.*

ACCESS

4. Two things that will catch you out

Role coverage answers the question you will actually be asked, which is never “what does this role mean” but “why can this person not get in”.



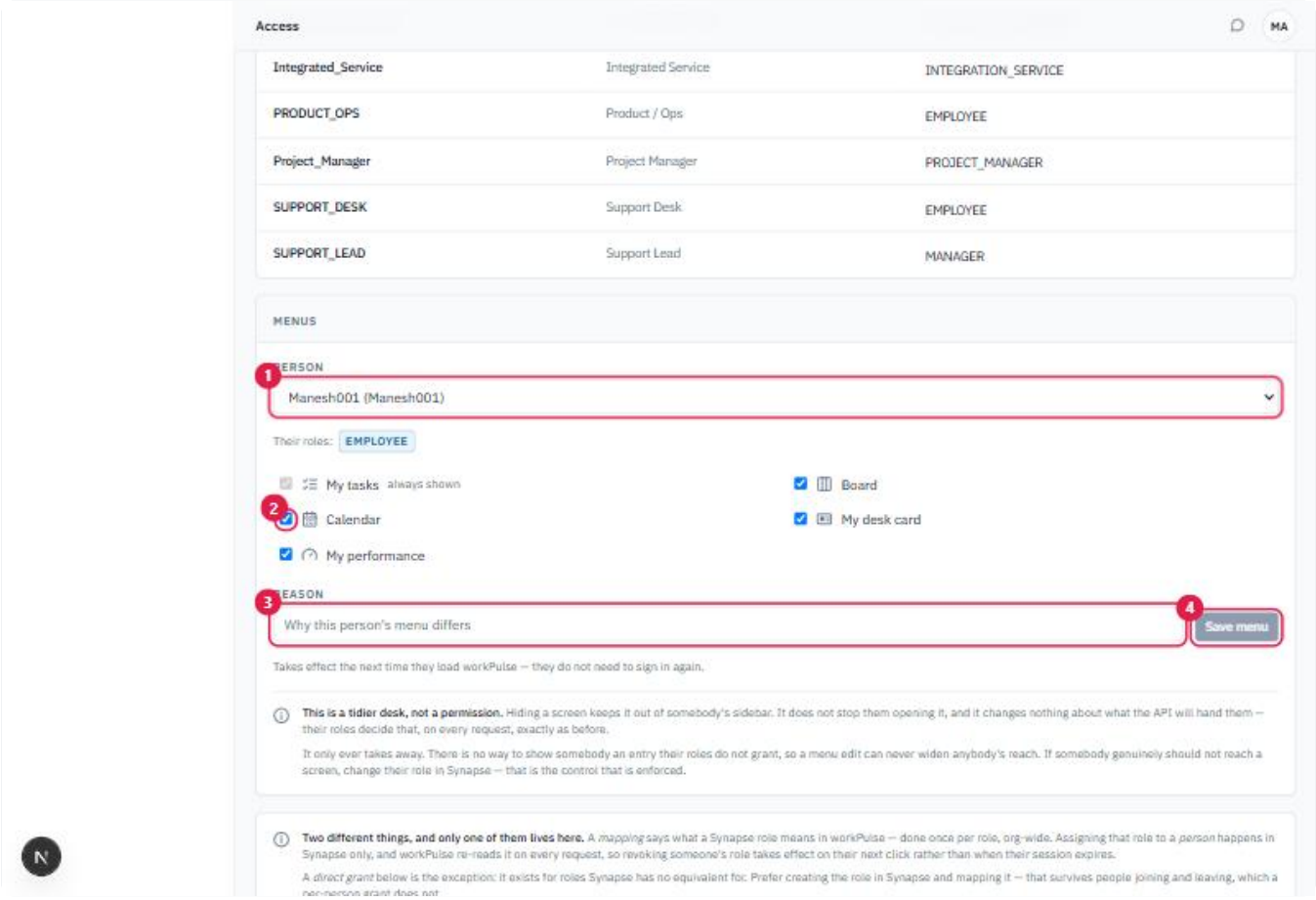
1. A role nobody can reach. Nothing maps to it and nobody holds it directly, so the controls behind it cannot be exercised by anyone. Identity is deny-by-default with no fallback, so this fails closed rather than over-granting — but it has to be fixed before go-live rather than discovered during it.
2. EMPLOYEE is reached by more Synapse roles than any other, because every member of staff holds it as a baseline whatever else they hold. workPulse takes the UNION of a person's mapped roles, so a further role only ever adds — a department head holds EMPLOYEE and DeptHead and loses nothing by it.
3. What a role is reached via. These mappings are the grant. Add one here to make it work today — and add it to ROLE_MAP in the deployment seed to make it survive, because the seed treats its own list as authoritative and deactivates every mapping not on it. The symptom otherwise is people who worked yesterday being refused today, with nothing in the release notes to explain it.

The trap this screen cannot show you: a mapped role still grants nothing unless it also carries the WorkPulse menu entry in Synapse AT THE USER'S OWN HIERARCHY. Synapse matches the menu against the hierarchy of the person, while its admin screen writes it at the hierarchy of the role — DeptHead sits at 0100 and ADMIN at 9999, so attaching the menu to either grants nothing to staff at 0200. That is the other reason everybody holds EMPLOYEE, which sits where the staff do.

ACCESS

5. Giving somebody a shorter menu

If a team should open workPulse and see four entries rather than fourteen, untick the rest. The list offered is exactly what that person's roles already grant.



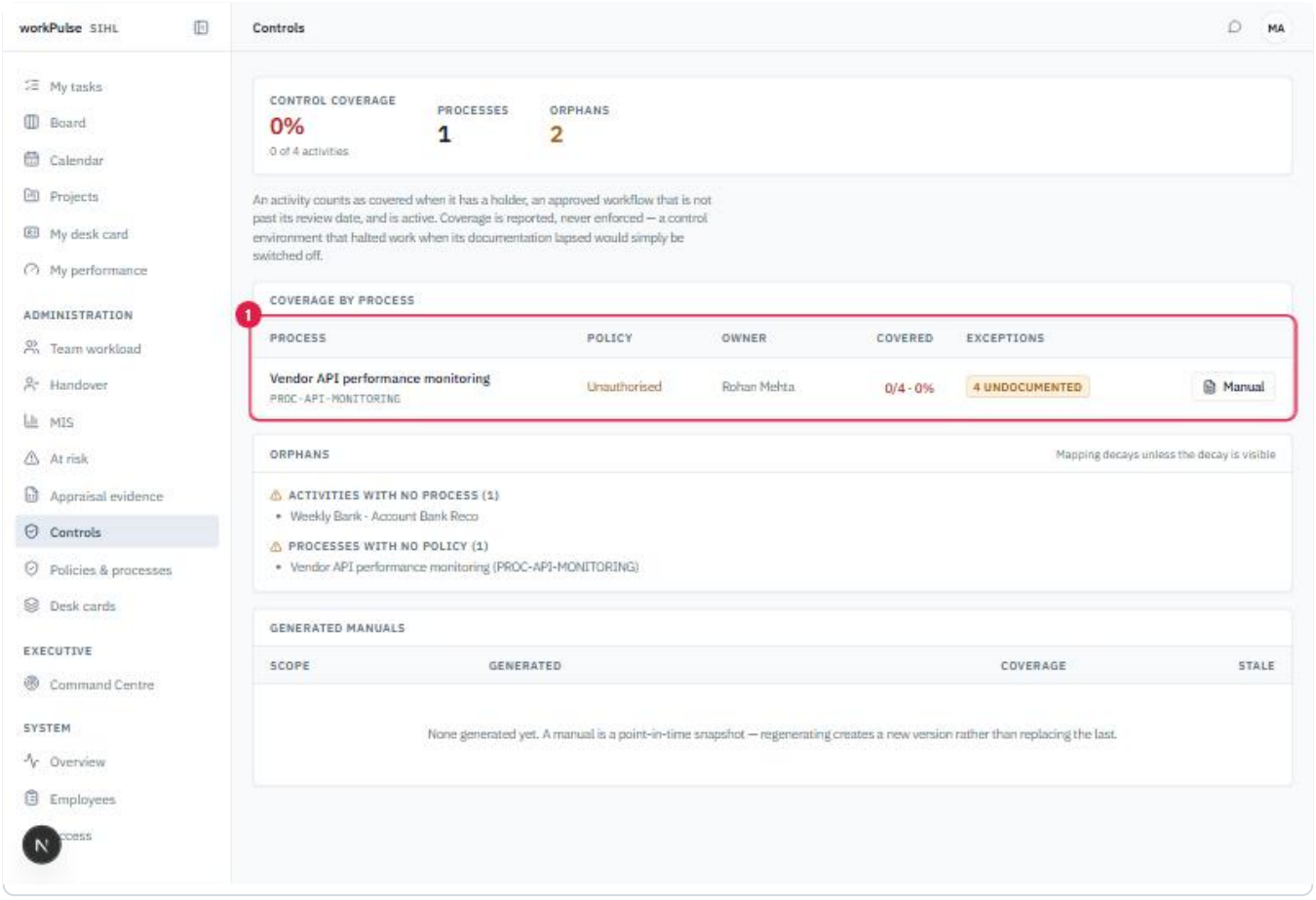
1. Who you are adjusting; their roles are shown beneath.
2. Untick to hide. Their own tasks can never be hidden.
3. Why. Recorded with the change.
4. Applies the next time they load workPulse — no re-login.

> *This is a tidier desk, not a permission. It cannot show anybody an entry their roles do not grant, and hiding one does not stop them opening it. If somebody genuinely must not reach a screen, change their role in Synapse — that is the control that is enforced.*

GOVERNANCE

6. What no control covers

The governance chain is desk card !’ activity !’ process !’ policy. This screen reports how much of it actually holds, and names what does not.



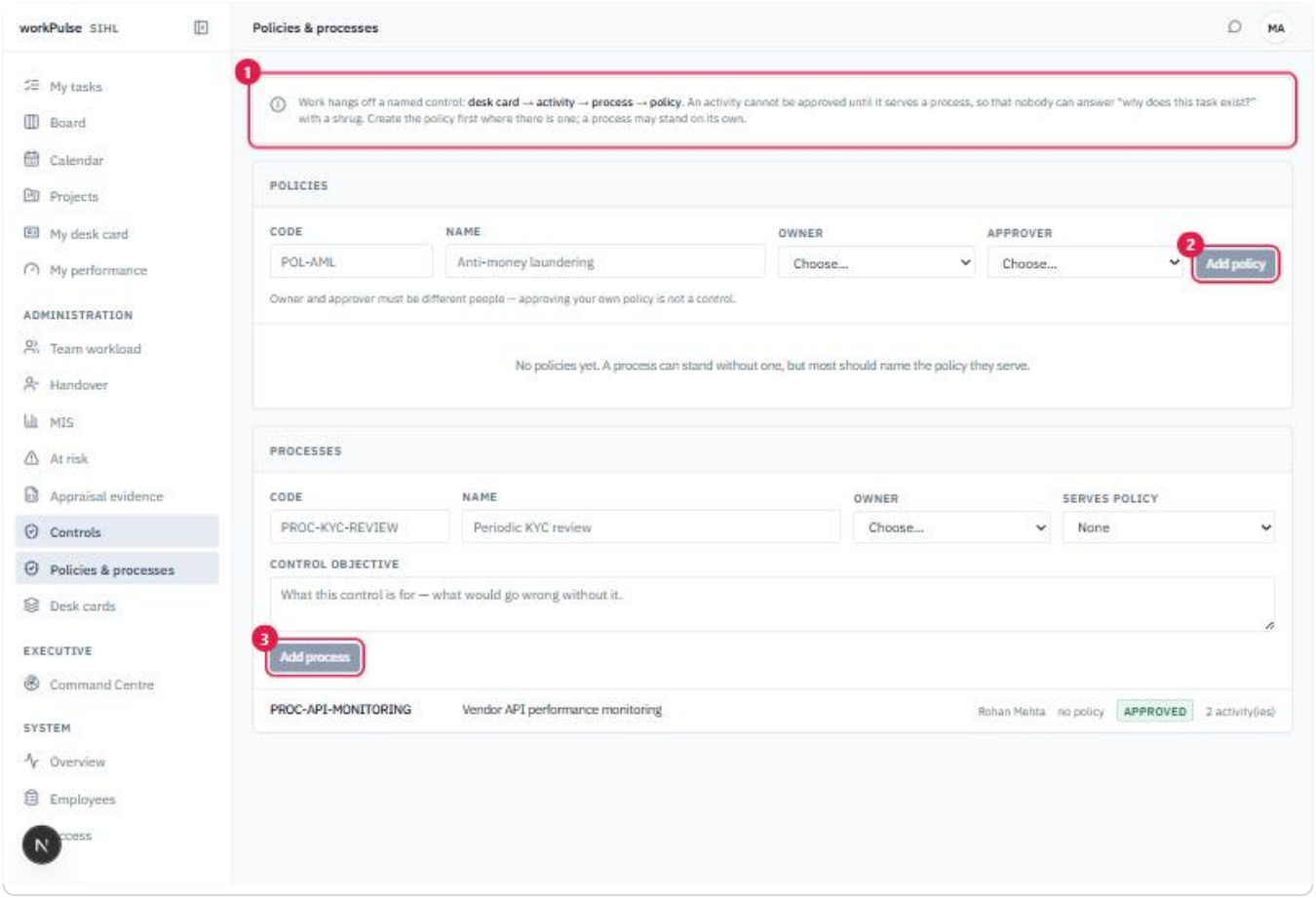
1. Coverage by process: the control, the policy it serves, its owner, and how much of the work claiming it is genuinely covered.
2. Orphans — activities serving no process, and processes serving no policy. Mapping decays unless the decay is visible, which is why this is a permanent panel rather than a one-off migration report.
3. Generated manuals: a point-in-time snapshot of the chain. Regenerating creates a new version rather than replacing the last.

> *Coverage is reported, never enforced retrospectively. A control environment that halted work the moment its documentation lapsed would simply be switched off, and then you would have neither the work nor the control.*

GOVERNANCE

7. Creating the controls

When a desk card activity has no process to serve, this is where one is created.

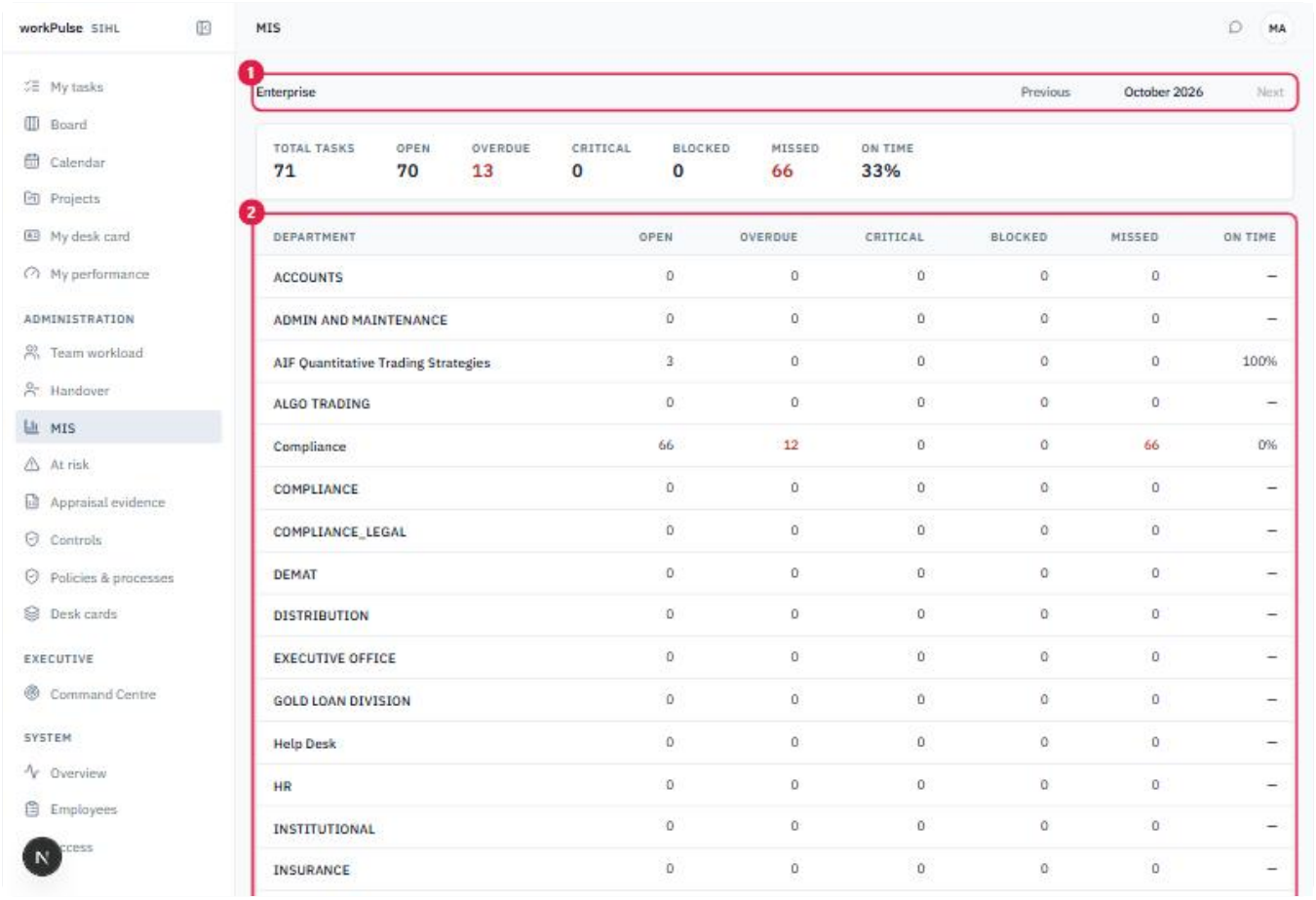


1. The chain, stated on the screen so nobody has to remember the order.
2. A policy. Its owner and approver must be different people — approving your own policy is not a control.
3. A process. It can stand without a policy, but most should name the one they serve.

OVERSIGHT

8. MIS

Four levels, each drillable into the one below and finally into the tasks themselves: enterprise, department, employee, task. Every figure above is traceable to the work that produced it.



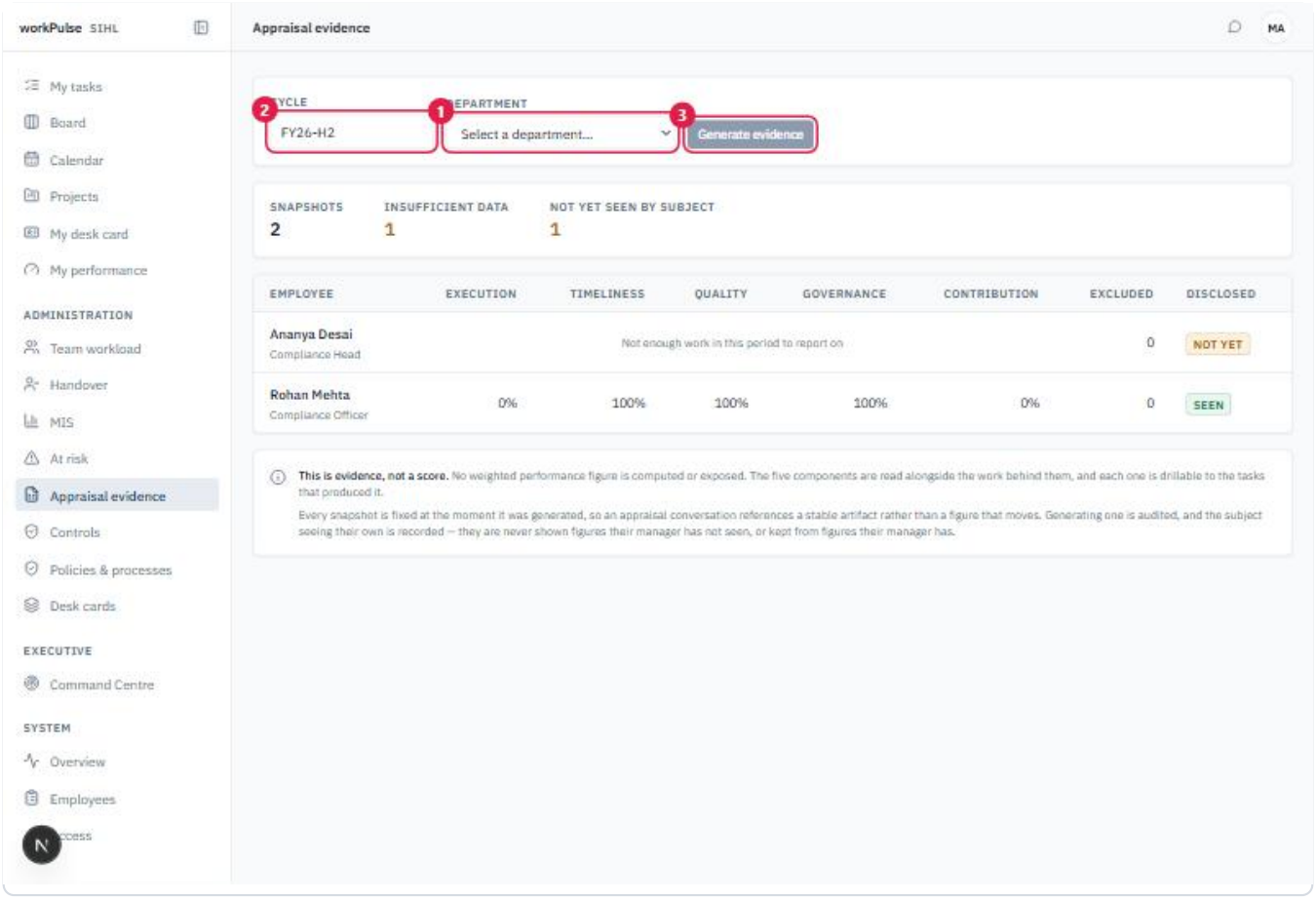
1. The period, and the level you are looking at.
2. By department. The scope is always disclosed — you are told what this view does and does not include rather than left to assume.

> *Figures here are operational, not a performance rating. Analytics about a person are visible to that person at the same time and in the same form as to their manager and HR; there is no manager-only view of somebody's numbers.*

OVERSIGHT

9. Evidence for HR

An appraisal snapshot per employee per cycle. Generating one is itself an audited act, because pulling somebody's performance evidence is an event worth recording.



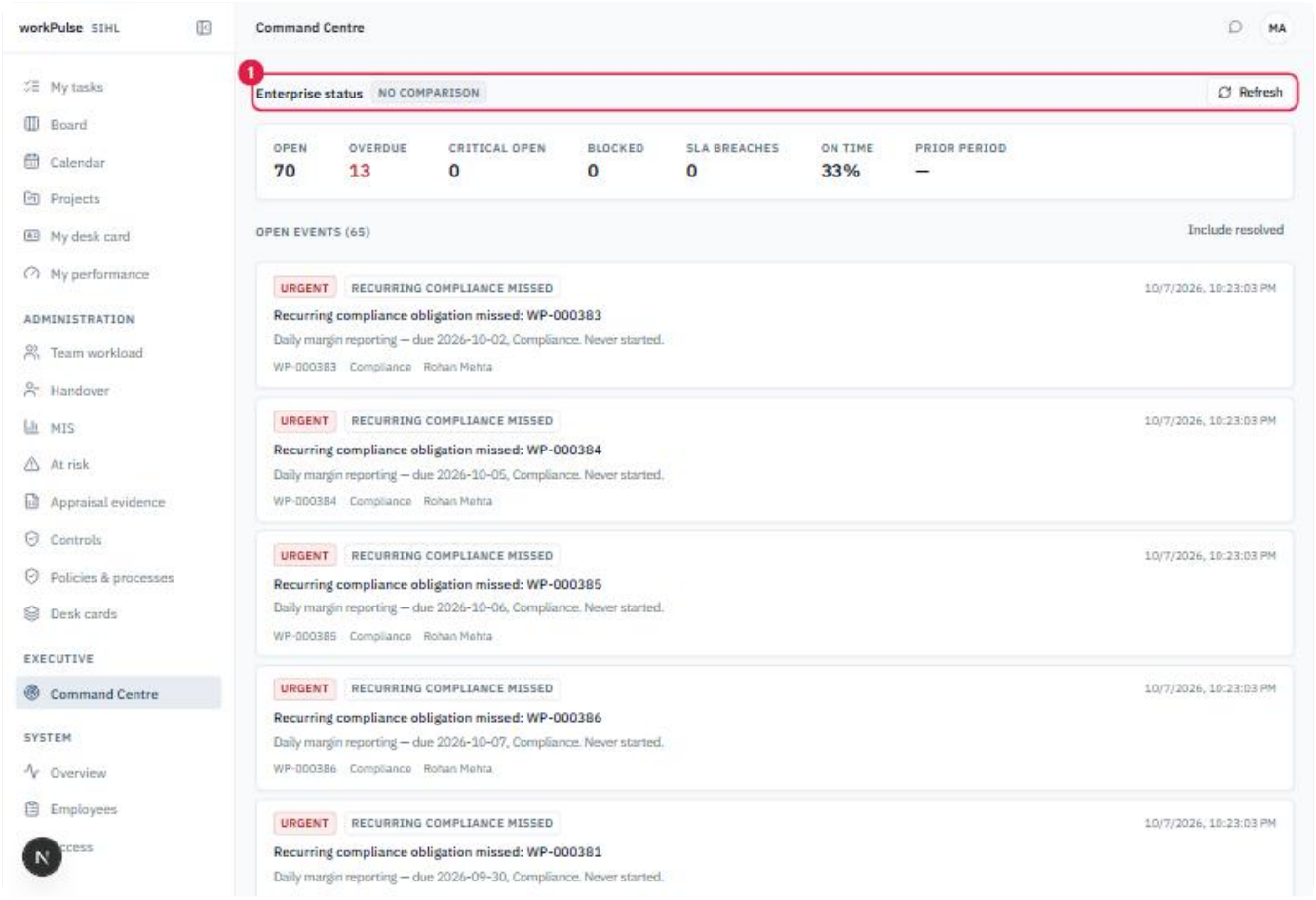
1. Pick the department.
2. HR's cycle label — opaque to workPulse, so it matches whatever HR already calls it.
3. Generate. The snapshot is frozen at that moment: an appraisal conversation must reference a fixed artifact, not a figure that moves between HR reading it and the employee being shown it.

> *Five components and deliberately no sixth combining them — this is evidence, not a score. Insufficient volume is reported as insufficient data, never as zero: somebody on extended leave and somebody performing badly must not look identical. "Seen by subject" is tracked, because analytics are never hidden from the person they describe.*

OVERSIGHT

10. Command Centre

The enterprise feed: missed recurring compliance obligations, SLA breaches, blocked work, each raised once and held open until resolved.



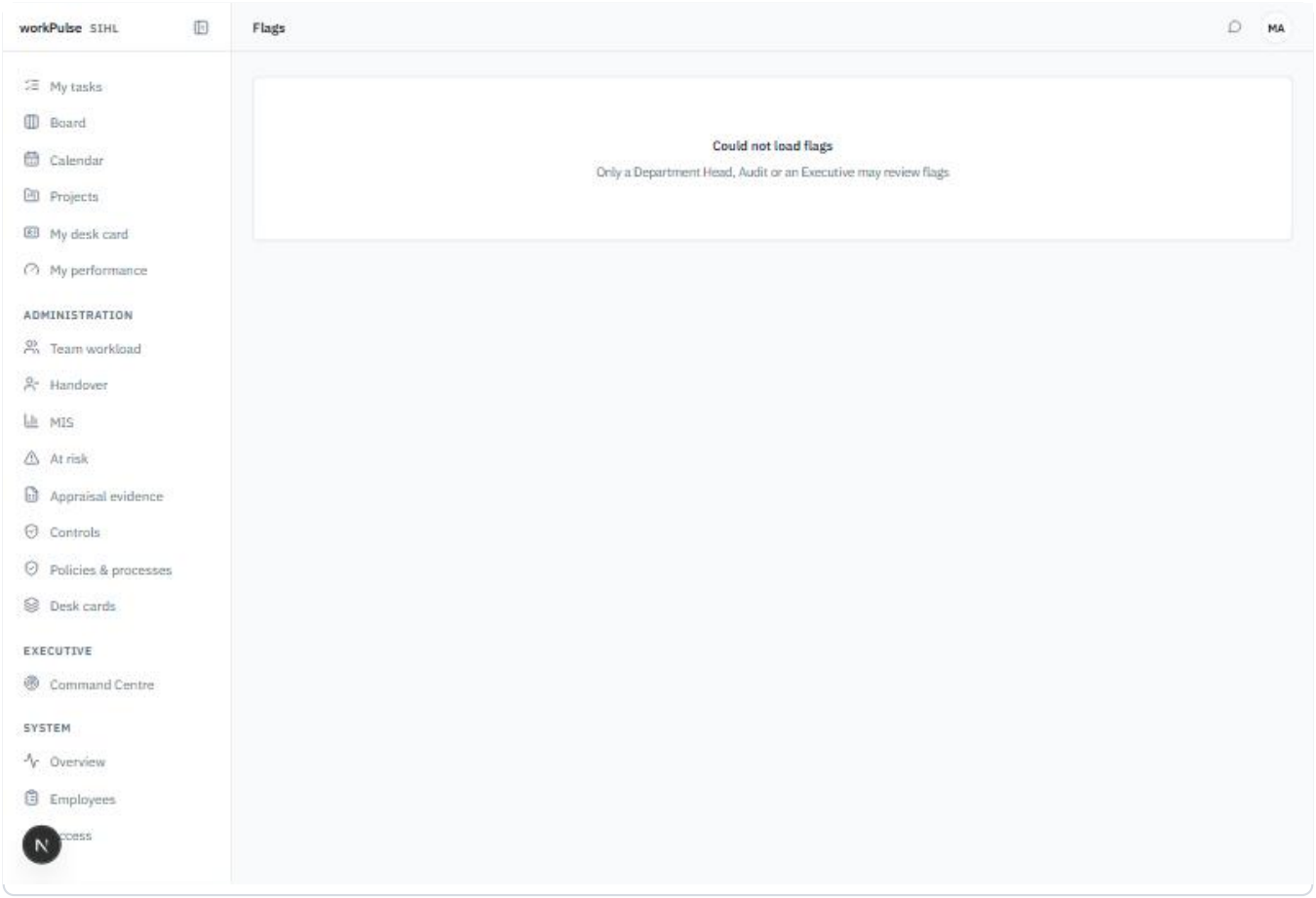
1. Enterprise status, with the prior period beside it for comparison.
2. The open events. Each names the task, the department and the person, and says what happened in a sentence.

> *Events are de-duplicated rather than re-raised: the same obligation missed on five days produces five distinct events, but one event is never raised twice for the same thing. If this list is long, that is a finding, not a bug.*

OVERSIGHT

11. Screens you cannot see — Flags

This is a system administrator being refused. Flags — the anti-gaming review queue — is for Department Heads, Audit and Executives. Administration is not oversight of people.



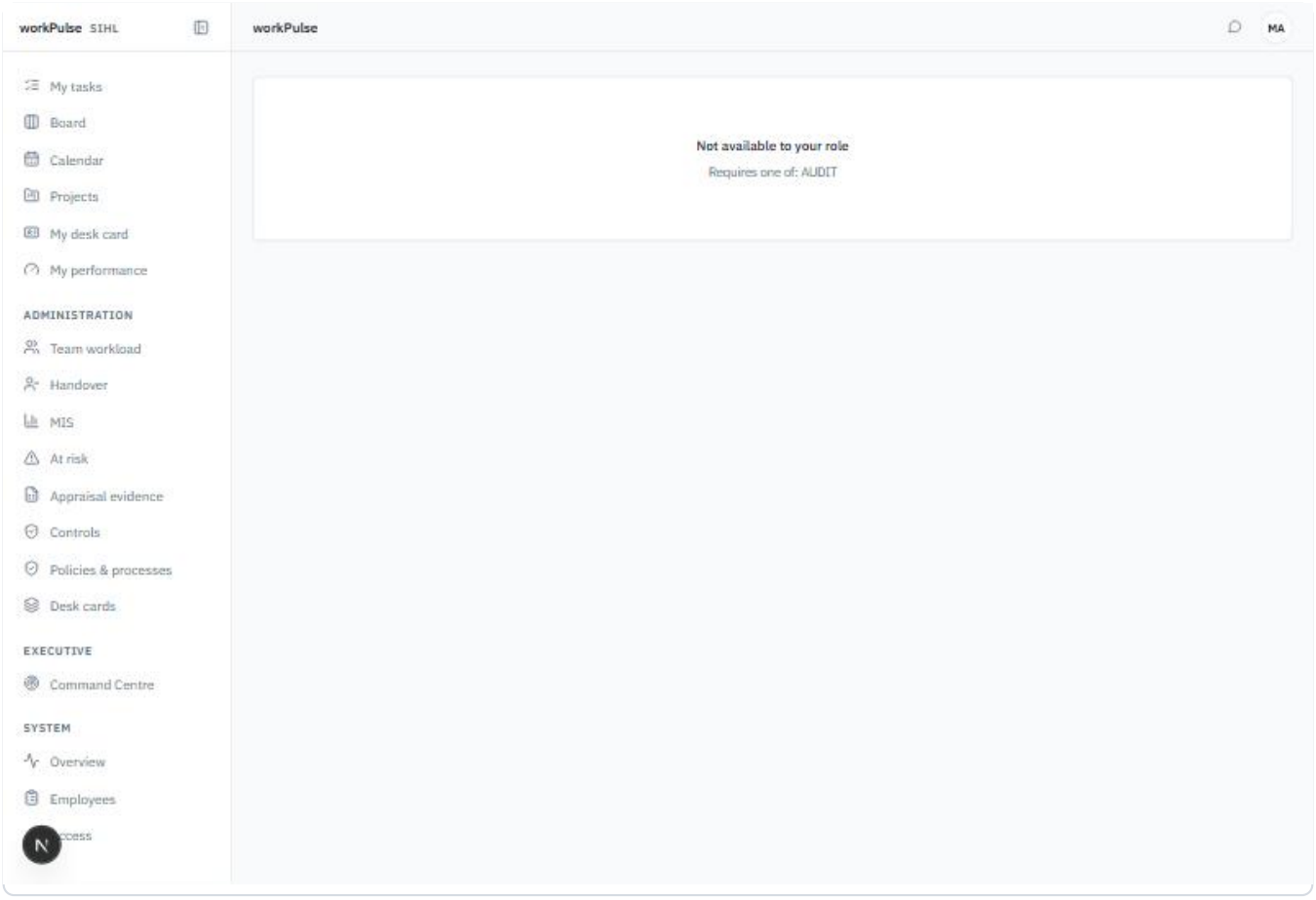
1. “Only a Department Head, Audit or an Executive may review flags.” The entry is also absent from your menu, so the refusal is consistent rather than a surprise.

Flags never penalise anybody automatically. Flagged work leaves both the numerator and the denominator until a human reviews it with a written reason — a heuristic that quietly docked somebody's numbers would be worse than having none, because it would be wrong invisibly.

OVERSIGHT

12. ...and the shadow quarter

The same again, more strictly: the shadow-quarter scoring review requires the Audit role and nothing else will do.



1 “Requires one of: AUDIT.” Not a system administrator, not an executive.

Audit is required to sit outside the line hierarchy, and a role that an administrator could grant themselves and then read would not be outside anything. If you need this view, the answer is a reviewed grant of the Audit role — recorded, and reportable to whoever asks who holds it.

RUNNING IT

13. Jobs, and what they do

Four things run on a schedule rather than when somebody clicks. If work stops appearing, this is the list to walk:

- Recurrence — generates tasks from approved desk card activities, honouring each activity's lead time. If a team's recurring work has stopped, check the card is approved and has a holder before suspecting the job.
- Frappe HR sync — refreshes the employee and department read-cache. Only this job writes those tables; nothing in the UI does.
- Anti-gaming — scans closures for missing evidence, reassignment patterns and trivial-task runs, and raises flags for a human to review.
- Command Centre and SLA prediction — raise events and assess open work hourly.
- There is also a role-mapping job for bulk Synapse mappings, and an attachment store on disk at ATTACHMENT_DIR which must be on a backed-up volume: lose it and the rows survive but the evidence does not.

SESSION_SECRET must be identical to Synapse's. If it drifts, every request fails authentication at once and the symptom looks like an outage rather than a configuration change.

RUNNING IT

14. Known gaps

Things an administrator will find before anybody reports them:

- The shadow-quarter screen is reachable only by typing the URL; it is not in the menu for anybody.
- A user whose Synapse role maps to nothing is told “Sign in to continue”, which is misleading — they are signed in, they have no role. Until the wording is fixed, read that message as “this person needs a mapping”.
- SLA targets have a screen, but almost nothing is configured in it yet. The At risk view stays quiet until somebody sets real targets — that is now a filling-in job rather than a missing feature.
- Attachments are written to local disk rather than object storage, so exactly one API instance may run. Two would each see only the files they wrote, and half of every evidence trail would look missing.
- A task can be added to a project, but not created from inside one in a single step; and there is no Gantt or timeline view.
- Three dependency vulnerabilities (multer, platform-express, lodash) need a NestJS major upgrade and are deliberately outstanding. Schedule it rather than letting it settle.

Four items that stood here on 18 October have since been built — the Employees page, raising and reassigning tasks, the SLA screen and the intake endpoint — and have been removed rather than left to mislead. None of what remains is a design problem.